

easy-db 网络层 API（Socket RESTful 调度器）开发指导书

一、模块概述

1.1 什么是网络层 API?

网络层 API 是指通过网络协议（如 TCP/HTTP）对外提供的编程接口，让远程开发者可以通过网络调用你的服务。

本次任务：在 `easy-db` 的 **Socket 通信层**上，实现一个简单的类似 **RESTful** 的请求调度器，让客户端可以通过 HTTP-like 的请求格式来操作数据库。

1.2 为什么要实现这个?

用途	说明
统一接口风格	使用 RESTful 风格（GET/POST/PUT/DELETE），更符合 Web 开发习惯
降低接入门槛	任何支持 HTTP 的客户端（浏览器、curl、Postman）都能访问
与现有系统集成	方便与其他服务（如前端页面、微服务）对接
理解底层原理	了解 HTTP 协议解析、路由分发、反射调用等底层机制

1.3 效果预览

```
# 使用 curl 发送 RESTful 请求
$ curl -X POST http://localhost:8080/api/v1/keys/user:001 -d "张三"
{"code":200,"message":"OK","data":"user:001"}

$ curl -X GET http://localhost:8080/api/v1/keys/user:001
{"code":200,"message":"OK","data":"张三"}

$ curl -X DELETE http://localhost:8080/api/v1/keys/user:001
{"code":200,"message":"OK","data":"user:001 deleted"}

$ curl -X GET http://localhost:8080/api/v1/keys
{"code":200,"message":"OK","data":["user:001","user:002"]}
```

二、核心需求

2.1 协议设计

2.1.1 请求格式

客户端发送的请求采用 **HTTP-like 文本格式**：

```
METHOD /path HTTP/1.1
Header1: value1
Header2: value2
Content-Length: xxx

{ "key": "value" }
```

简化版（最低要求）：

```
METHOD /path  
参数...
```

完整版（推荐）：

```
GET /api/v1/keys/user:001 HTTP/1.1  
Host: localhost:8080
```

2.1.2 响应格式

服务端返回 **JSON** 格式的响应：

```
// 成功响应  
{  
  "code": 200,  
  "message": "OK",  
  "data": "张三"  
}  
  
// 列表响应  
{  
  "code": 200,  
  "message": "OK",  
  "data": [ "key1", "key2", "key3" ]  
}  
  
// 错误响应  
{  
  "code": 404,  
  "message": "Key not found",  
  "data": null  
}
```

2.2 路由设计

2.2.1 路由映射表

将 RESTful 路径映射到具体的操作：

HTTP 方法	路径	功能	对应 DB 命令
POST	/api/v1/keys/{key}	存储键值对	SET {key} {value}
GET	/api/v1/keys/{key}	获取键值	GET {key}
DELETE	/api/v1/keys/{key}	删除键值	DEL {key}
GET	/api/v1/keys	列出所有键	KEYS *
GET	/api/v1/keys?pattern={pattern}	列出匹配的键	KEYS {pattern}
GET	/api/v1/exists/{key}	检查键是否存在	EXISTS {key}
DELETE	/api/v1/flush	清空所有数据	FLUSH

2.2.2 路径参数

- 路径中 {key} 表示动态参数
- 例如 /api/v1/keys/user:001 中的 user:001 就是 key 参数

2.3 请求解析

需求 2.3.1：请求行解析

解析客户端发送的第一行请求：

```
// 示例输入: "GET /api/v1/keys/user:001 HTTP/1.1"
// 解析结果: method="GET", path="/api/v1/keys/user:001", version="HTTP/1.1"

public class HttpRequest {
    private String method;
    private String path;
    private String version;
    private Map<String, String> headers;
    private String body;

    public static HttpRequest parse(String request) {
        String[] lines = request.split("\n");
        // 解析请求行
        String[] firstLine = lines[0].trim().split(" ");
        // ...
    }
}
```

需求 2.3.2：请求体解析

对于 POST 请求，从请求体中提取 JSON 数据：

```
// 请求体示例: {"value":"张三"}
// 提取 value 字段的值

private String parseBodyForValue(String body) {
    try {
        JSONObject json = new JSONObject(body);
        return json.getString("value");
    } catch (Exception e) {
        return null;
    }
}
```

2.4 调度器实现

需求 2.4.1: 路由注册

```
public class RestDispatcher {
    private Map<String, RouteHandler> routes = new HashMap<>();

    public void register(String method, String path, RouteHandler handler) {
        String key = method + " " + path;
        routes.put(key, handler);
    }

    public RouteHandler findHandler(String method, String path) {
        // 1. 精确匹配
        String key = method + " " + path;
        if (routes.containsKey(key)) {
            return routes.get(key);
        }
        // 2. 路径参数匹配 (如 /api/v1/keys/{key})
        // ...
        return null;
    }
}
```

需求 2.4.2: 处理器接口

```
@FunctionalInterface
public interface RouteHandler {
    HttpResponse handle(HttpRequest request);
}
```

需求 2.4.3: 处理器实现

```
// GET /api/v1/keys/{key} 处理器
public class GetKeyHandler implements RouteHandler {
    private NormalStore store;

    public GetKeyHandler(NormalStore store) {
        this.store = store;
    }

    @Override
    public HttpResponse handle(HttpRequest request) {
```

```

String key = extractPathParam(request.getPath(), "/api/v1/keys/");
String value = store.get(key);

if (value == null) {
    return HttpResponse.notFound("Key not found: " + key);
}
return HttpResponse.ok(value);
}
}

```

2.5 Socket 服务端集成

将 HTTP 调度器集成到现有的 `SocketServerController` 中：

```

public class SocketServerController {
    private RestDispatcher dispatcher;
    private NormalStore store;

    public SocketServerController(int port) {
        this.store = new NormalStore();
        this.dispatcher = new RestDispatcher();

        // 注册路由
        registerRoutes();
    }

    private void registerRoutes() {
        // POST /api/v1/keys/{key}
        dispatcher.register("POST", "/api/v1/keys/{key}", new PostKeyHandler(store));

        // GET /api/v1/keys/{key}
        dispatcher.register("GET", "/api/v1/keys/{key}", new GetKeyHandler(store));

        // DELETE /api/v1/keys/{key}
        dispatcher.register("DELETE", "/api/v1/keys/{key}", new DeleteKeyHandler(store));

        // GET /api/v1/keys
        dispatcher.register("GET", "/api/v1/keys", new ListKeysHandler(store));
    }
}

```

三、完整代码实现

3.1 包结构

```

src/main/java/
├── controller/
│   ├── SocketServerController.java # 已有（需扩展）
│   └── http/ # ← 新增 HTTP 相关
│       ├── HttpRequest.java # HTTP 请求解析
│       ├── HttpResponse.java # HTTP 响应构建
│       ├── RestDispatcher.java # REST 调度器
│       ├── RouteHandler.java # 处理器接口
│       └── handler/ # 具体处理器
│           ├── GetKeyHandler.java
│           └── PostKeyHandler.java

```

```
| |
| |
| | DeleteKeyHandler.java
| | ListKeysHandler.java
| ...
```

3.2 核心类实现

HttpRequest.java

```
package controller.http;

import java.io.BufferedReader;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class HttpRequest {
    private String method;
    private String path;
    private String version;
    private Map<String, String> headers;
    private String body;

    private HttpRequest() {
        this.headers = new HashMap<>();
    }

    /**
     * 从 Socket 输入流解析 HTTP 请求
     */
    public static HttpRequest parse(BufferedReader reader) throws IOException {
        HttpRequest request = new HttpRequest();

        // 1. 解析请求行
        String firstLine = reader.readLine();
        if (firstLine == null || firstLine.isEmpty()) {
            return null;
        }
        String[] parts = firstLine.split(" ");
        if (parts.length < 2) {
            return null;
        }
        request.method = parts[0];
        request.path = parts[1];
        request.version = parts.length > 2 ? parts[2] : "HTTP/1.1";

        // 2. 解析请求头
        String line;
        int contentLength = 0;
        while (!(line = reader.readLine()).isEmpty()) {
            String[] headerParts = line.split(": ", 2);
            if (headerParts.length == 2) {
                request.headers.put(headerParts[0], headerParts[1]);
                if (headerParts[0].equalsIgnoreCase("Content-Length")) {
                    contentLength = Integer.parseInt(headerParts[1]);
                }
            }
        }
    }
}
```

```

// 3. 解析请求体
if (contentLength > 0) {
    char[] bodyChars = new char[contentLength];
    reader.read(bodyChars, 0, contentLength);
    request.body = new String(bodyChars);
}

return request;
}

// Getter 方法...
public String getMethod() { return method; }
public String getPath() { return path; }
public String getBody() { return body; }
public String getHeader(String name) { return headers.get(name); }
public Map<String, String> getHeaders() { return headers; }

/**
 * 从路径中提取参数
 * 例如: /api/v1/keys/user:001 → user:001
 */
public String getPathParam(String prefix) {
    if (path.startsWith(prefix)) {
        return path.substring(prefix.length());
    }
    return null;
}

/**
 * 解析查询参数
 * 例如: /api/v1/keys?pattern=user:* → {"pattern": "user:*"}
 */
public Map<String, String> getQueryParams() {
    Map<String, String> params = new HashMap<>();
    if (!path.contains("?")) return params;

    String queryString = path.substring(path.indexOf("?") + 1);
    String[] pairs = queryString.split("&");
    for (String pair : pairs) {
        String[] kv = pair.split("=", 2);
        if (kv.length == 2) {
            params.put(kv[0], kv[1]);
        }
    }
    return params;
}
}

```

HttpResponse.java

```

package controller.http;

import com.alibaba.fastjson.JSONObject;

public class HttpResponse {
    private int statusCode;
    private String statusMessage;
}

```

```

private Object data;

private HttpResponse(int statusCode, String statusMessage, Object data) {
    this.statusCode = statusCode;
    this.statusMessage = statusMessage;
    this.data = data;
}

// 成功响应
public static HttpResponse ok(Object data) {
    return new HttpResponse(200, "OK", data);
}

// 创建成功
public static HttpResponse created(Object data) {
    return new HttpResponse(201, "Created", data);
}

// 未找到
public static HttpResponse notFound(String message) {
    return new HttpResponse(404, "Not Found", message);
}

// 错误响应
public static HttpResponse error(int code, String message) {
    return new HttpResponse(code, message, null);
}

/**
 * 转换为 HTTP 响应字符串
 */
public String toHttpString() {
    JSONObject json = new JSONObject();
    json.put("code", statusCode);
    json.put("message", statusMessage);
    json.put("data", data);

    String jsonBody = json.toJSONString();
    return String.format(
        "HTTP/1.1 %d %s\r\n" +
        "Content-Type: application/json\r\n" +
        "Content-Length: %d\r\n" +
        "Access-Control-Allow-Origin: *\r\n" +
        "\r\n" +
        "%s",
        statusCode, statusMessage,
        jsonBody.getBytes().length,
        jsonBody
    );
}
}

```

RouteHandler.java


```

package controller.http;

@FunctionalInterface
public interface RouteHandler {
    HttpResponse handle(HttpRequest request);
}

```

RestDispatcher.java

```

package controller.http;

import java.util.HashMap;
import java.util.Map;

public class RestDispatcher {
    private Map<String, RouteHandler> exactRoutes = new HashMap<>();
    private Map<String, RouteHandler> patternRoutes = new HashMap<>();

    /**
     * 注册路由
     * @param method HTTP 方法 (GET/POST/PUT/DELETE)
     * @param path 路径 (支持 {key} 作为路径参数)
     * @param handler 处理器
     */
    public void register(String method, String path, RouteHandler handler) {
        if (path.contains("{") && path.contains("}")) {
            // 带参数的路由, 转换为正则
            String pattern = path.replaceAll("\\{[^}]+\\}", "([^/]+)");
            patternRoutes.put(method + ":" + pattern, handler);
        } else {
            exactRoutes.put(method + ":" + path, handler);
        }
    }

    /**
     * 查找匹配的路由
     */
    public RouteHandler findHandler(String method, String path) {
        // 1. 精确匹配
        String exactKey = method + ":" + path;
        if (exactRoutes.containsKey(exactKey)) {
            return exactRoutes.get(exactKey);
        }

        // 2. 模式匹配 (路径参数)
        String pathWithoutQuery = path.split("\\?")[0];
        for (Map.Entry<String, RouteHandler> entry : patternRoutes.entrySet()) {
            String pattern = entry.getKey().split(":", 2)[1];
            if (pathWithoutQuery.matches(pattern.replaceAll("\\{[^}]+\\}", "([^/]+)"))) {
                return entry.getValue();
            }
        }

        return null;
    }

    /**

```

```

    * 提取路径参数值
    */
    public Map<String, String> extractPathParams(String path, String routePattern) {
        Map<String, String> params = new HashMap<>();
        String[] pathParts = path.split("\\?")[0].split("/");
        String[] patternParts = routePattern.split("/");

        for (int i = 0; i < patternParts.length && i < pathParts.length; i++) {
            if (patternParts[i].startsWith("{") && patternParts[i].endsWith("}")) {
                String paramName = patternParts[i].substring(1, patternParts[i].length() - 1);
                params.put(paramName, pathParts[i]);
            }
        }
        return params;
    }
}

```

PostKeyHandler.java

```

package controller.http.handler;

import com.alibaba.fastjson.JSONObject;
import controller.http.HttpRequest;
import controller.http.HttpResponse;
import controller.http.RouteHandler;
import service.NormalStore;

public class PostKeyHandler implements RouteHandler {
    private NormalStore store;

    public PostKeyHandler(NormalStore store) {
        this.store = store;
    }

    @Override
    public HttpResponse handle(HttpRequest request) {
        // 1. 从路径中提取 key
        String key = request.getPathParam("/api/v1/keys/");
        if (key == null || key.isEmpty()) {
            return HttpResponse.error(400, "Missing key in path");
        }

        // 2. 从请求体中提取 value
        String body = request.getBody();
        if (body == null || body.isEmpty()) {
            return HttpResponse.error(400, "Missing request body");
        }

        try {
            JSONObject json = JSONObject.parseObject(body);
            String value = json.getString("value");
            if (value == null) {
                return HttpResponse.error(400, "Missing 'value' field in JSON body");
            }

            // 3. 执行存储
            store.set(key, value);
            return HttpResponse.created(key);
        }
    }
}

```

```

    } catch (Exception e) {
        return HttpResponse.error(500, "Invalid JSON body: " + e.getMessage());
    }
}
}

```

SocketServerController 修改

```

package controller;

import controller.http.HttpRequest;
import controller.http.HttpResponse;
import controller.http.RestDispatcher;
import controller.http.RouteHandler;
import controller.http.handler.*;
import service.NormalStore;

import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class SocketServerController {
    private ServerSocket serverSocket;
    private ExecutorService threadPool;
    private NormalStore store;
    private RestDispatcher dispatcher;

    public SocketServerController(int port) throws IOException {
        this.serverSocket = new ServerSocket(port);
        this.threadPool = Executors.newCachedThreadPool();
        this.store = new NormalStore();
        this.dispatcher = new RestDispatcher();

        // 注册 REST 路由
        registerRoutes();
    }

    private void registerRoutes() {
        // POST /api/v1/keys/{key}
        dispatcher.register("POST", "/api/v1/keys/{key}", new PostKeyHandler(store));

        // GET /api/v1/keys/{key}
        dispatcher.register("GET", "/api/v1/keys/{key}", new GetKeyHandler(store));

        // DELETE /api/v1/keys/{key}
        dispatcher.register("DELETE", "/api/v1/keys/{key}", new DeleteKeyHandler(store));

        // GET /api/v1/keys
        dispatcher.register("GET", "/api/v1/keys", new ListKeysHandler(store));
    }

    public void start() {
        System.out.println("🚀 easy-db RESTful Server started on port " +
            serverSocket.getLocalPort());
        System.out.println("🔗 Available endpoints:");
    }
}

```

```

System.out.println("    POST    /api/v1/keys/{key}    - Store a key-value pair");
System.out.println("    GET     /api/v1/keys/{key}    - Get value by key");
System.out.println("    DELETE  /api/v1/keys/{key}    - Delete a key");
System.out.println("    GET     /api/v1/keys           - List all keys");

while (true) {
    try {
        Socket client = serverSocket.accept();
        threadPool.submit(new HttpRequestHandler(client, dispatcher));
    } catch (IOException e) {
        e.printStackTrace();
    }
}

/**
 * HTTP 请求处理器
 */
private static class HttpRequestHandler implements Runnable {
    private Socket client;
    private RestDispatcher dispatcher;

    public HttpRequestHandler(Socket client, RestDispatcher dispatcher) {
        this.client = client;
        this.dispatcher = dispatcher;
    }

    @Override
    public void run() {
        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(client.getInputStream()));
            PrintWriter writer = new PrintWriter(client.getOutputStream(), true)) {

            // 1. 解析 HTTP 请求
            HttpRequest request = HttpRequest.parse(reader);
            if (request == null) {
                writer.println(HttpResponse.error(400, "Invalid request").toHttpString());
                return;
            }

            System.out.println("📝 " + request.getMethod() + " " + request.getPath());

            // 2. 查找路由
            RouteHandler handler = dispatcher.findHandler(request.getMethod(), request.getPath());
            if (handler == null) {
                writer.println(HttpResponse.error(404, "Route not found: " +
request.getPath()).toHttpString());
                return;
            }

            // 3. 执行处理
            HttpResponse response = handler.handle(request);

            // 4. 返回响应
            writer.println(response.toHttpString());

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```
    }  
    }  
}
```

四、使用示例

4.1 启动服务端

```
public class RestServerUsage {  
    public static void main(String[] args) throws IOException {  
        SocketServerController server = new SocketServerController(8080);  
        server.start();  
    }  
}
```

4.2 使用 curl 调用

```
# 1. 存储数据 (POST)  
$ curl -X POST http://localhost:8080/api/v1/keys/user:001 \  
    -H "Content-Type: application/json" \  
    -d '{"value":"张三"}'  
{ "code":201,"message":"Created","data":"user:001"}  
  
$ curl -X POST http://localhost:8080/api/v1/keys/user:002 \  
    -H "Content-Type: application/json" \  
    -d '{"value":"李四"}'  
{ "code":201,"message":"Created","data":"user:002"}  
  
$ curl -X POST http://localhost:8080/api/v1/keys/product:001 \  
    -H "Content-Type: application/json" \  
    -d '{"value":"手机"}'  
{ "code":201,"message":"Created","data":"product:001"}  
  
# 2. 查询数据 (GET)  
$ curl -X GET http://localhost:8080/api/v1/keys/user:001  
{ "code":200,"message":"OK","data":"张三"}  
  
$ curl -X GET http://localhost:8080/api/v1/keys/user:999  
{ "code":404,"message":"Not Found","data":"Key not found: user:999"}  
  
# 3. 列出所有键  
$ curl -X GET http://localhost:8080/api/v1/keys  
{ "code":200,"message":"OK","data":["user:001","user:002","product:001"]}  
  
# 4. 删除数据 (DELETE)  
$ curl -X DELETE http://localhost:8080/api/v1/keys/user:002  
{ "code":200,"message":"OK","data":"user:002 deleted"}  
  
$ curl -X GET http://localhost:8080/api/v1/keys  
{ "code":200,"message":"OK","data":["user:001","product:001"]}
```

4.3 使用 Java 的 HttpURLConnection 调用

```
import java.net.HttpURLConnection;
```

```

import java.net.URL;
import java.io.OutputStream;
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class RestClientExample {
    public static void main(String[] args) throws Exception {
        // 1. POST 请求 (存储)
        URL url = new URL("http://localhost:8080/api/v1/keys/user:003");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setDoOutput(true);

        String jsonBody = "{\"value\":\"王五\"}";
        try (OutputStream os = conn.getOutputStream()) {
            os.write(jsonBody.getBytes());
            os.flush();
        }

        int responseCode = conn.getResponseCode();
        System.out.println("Response Code: " + responseCode);

        // 读取响应
        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(conn.getInputStream()))) {
            String line;
            StringBuilder response = new StringBuilder();
            while ((line = reader.readLine()) != null) {
                response.append(line);
            }
            System.out.println("Response: " + response);
        }
        conn.disconnect();
    }
}

```

4.4 使用浏览器访问

直接在浏览器地址栏输入：

```
http://localhost:8080/api/v1/keys/user:001
```

浏览器会显示 JSON 响应。

五、开发任务清单

阶段	任务	状态
阶段一	创建 <code>http</code> 包, 定义 <code>HttpRequest</code> 和 <code>HttpResponse</code>	☐
阶段二	实现 <code>HttpRequest.parse()</code> 解析请求	☐
阶段三	实现 <code>HttpResponse.toHttpString()</code> 生成响应	☐
阶段四	创建 <code>RouteHandler</code> 接口和 <code>RestDispatcher</code>	☐
阶段五	实现各处理器 (<code>GetKey/PostKey/DeleteKey/ListKeys</code>)	☐
阶段六	修改 <code>SocketServerController</code> 集成调度器	☐
阶段七	编写 REST 客户端示例代码	☐
阶段八 (选做)	支持路径参数提取 (<code>{key}</code>)	☐
阶段九 (选做)	支持查询参数 (<code>?pattern=user:*</code>)	☐

六、常见问题

Q1: Socket 层解析 HTTP 请求时, 如何知道请求体结束?

A: 通过 `Content-Length` 请求头判断。先读取请求头, 获取 `Content-Length` 值, 然后精确读取指定长度的字节。

Q2: 路径参数匹配如何实现?

A: 在注册路由时, 将 `{key}` 替换为正则表达式 `([^\s/]+)`, 匹配时提取对应的值。具体实现见 `RestDispatcher` 类。

Q3: 如何支持中文请求?

A: 确保使用 UTF-8 编码:

```
new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8)
```

Q4: POST 请求的 JSON 体如何解析?

A: 使用 `fastjson` 或 `Gson` 库:

```
<dependency>
  <groupId>com.alibaba</groupId>
  <artifactId>fastjson</artifactId>
  <version>1.2.83</version>
</dependency>
```

Q5: 浏览器直接访问报跨域错误?

A: 在响应中添加 CORS 头:

```
writer.println("Access-Control-Allow-Origin: *");
writer.println("Access-Control-Allow-Methods: GET, POST, DELETE, OPTIONS");
```

七、提交要求

1. 代码: HTTP 调度器模块全部源码

2. 文档：在课程设计报告中增加 **RESTful API** 章节，内容包括：
 - API 设计文档（每个端点的说明）
 - 使用示例截图（至少 5 张，含 curl 和浏览器）
 - 路由匹配实现思路
3. 演示：验收时现场演示 API 调用

八、参考资料

- [HTTP/1.1 协议规范](#)
 - [RESTful API 设计最佳实践](#)
 - [Java Socket 编程](#)
 - [JSON 格式标准](#)
-

祝同学们顺利完成网络层 API 开发！